

Politecnico di Milano

Anno accademico 2012-2013

Ingegneria del Software – Appello del 19 luglio 2012

Cognome:		
Nome:		Matricola:
Sezione (segnarne una): ☐ Baresi	☐ Ghezzi	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.

Esercizio 1:

Esercizio 2:

Esercizio 3:

Esercizio 4:

Esercizio 1

Si scrivano in JML le pre- e post- condizioni del seguente metodo **Substrings**. Questo prende in ingresso due array di caratteri x e y e restituisce, in un array di interi, l'elenco, in ordine qualunque, di tutti e soli gli indici in cui y compare come sottostringa di x. Ad esempio, se $x = [abbbbbabbbabbbabb]$ e $y = [abb]$, il metodo restituisce l'array $[0\ 6\ 10\ 14]$.

Soluzione:

```
//@assignable \nothing;
//@requires x!=null && y!=null;
//@ensures \result != null &&
//   \result contiene solo indici da cui inizia
//   una comparsa di y in x:
//@  (\forall int i; 0 \le i && i< \result.length;
//@      (\forall int j; 0\le j && j < y.length;
//@           \result[i]+j< x.length && x[\result[i]+j]== y[j])) &&
//   \result contiene tutti gli indici in cui compare y:
//@  (\forall int i; 0 \le i && i< x.length;
//@      (\forall int j; 0\le j && j < y.length;
//@           i+j< x.length && x[i+j]== y[j])
//@      ==>
//@      (\exists k; 0 \le k && k<\result.length; \result[k]==i));
public static int[] substrings(char[] x,char[] y)
```

Esercizio 2

Si consideri un distributore di caramelle. Il costo di ogni caramella è di una singola moneta da 50 cents. Il controllo del distributore è implementato in Java, con codice che include fra l'altro la seguente classe, qui specificata solo parzialmente:

```
public class Distributore {
  public final int MONETA = 50;
  public static final int ESAURITO=0;
  public static final int PRONTO=1;
  public static final int VENDUTO=2;
  public static final int FORNITO=3;

  //@requires caramelleIniziali>0;
  //@ensures caramelle() == caramelleIniziali;
  //@ensures monete() == 0;
  //@ensures stato()==PRONTO;
  public Distributore (int caramelleIniziali);

  public void inserisciMoneta();

  public int restituisciMoneta();

  public void giraManopola();

  public void preleva();

  //@ensures (* rilascia una caramella *);
  public void rilascia();

  //@requires nuoveCaramelle>0;
  //@ensures caramelle() == nuoveCaramelle && stato==PRONTO;
```

```
  public void riempi(int nuoveCaramelle);

  //@ensures (*\result e' il numero di caramelle rimaste*);
  public /*@ pure @*/ caramelle();

  //@ensures (*\result e' il numero di monete nel distributore*);
  public /*@ pure @*/ int numMonete();

  //@ensures (*\result e' lo stato (ESAURITO, PRONTO, ecc.)
  //@           in cui si trova il distributore; *)
  public /*@ pure @*/ int stato();

}
```

I metodi inserisciMoneta(), restituisciMoneta(), giraManopola() e preleva() corrispondono alle quattro possibili azioni dell'utente sul distributore, mentre gli altri metodi rappresentano funzionalità disponibili al codice Java del distributore. Il metodo rilascia() (che non corrisponde a un evento disponibile all'utilizzatore) si interfaccia con il dispositivo in modo da ordinare il rilascio fisico della caramella, mentre riempi() corrisponde al riempimento del deposito delle caramelle (e può essere chiamato solo dal gestore).

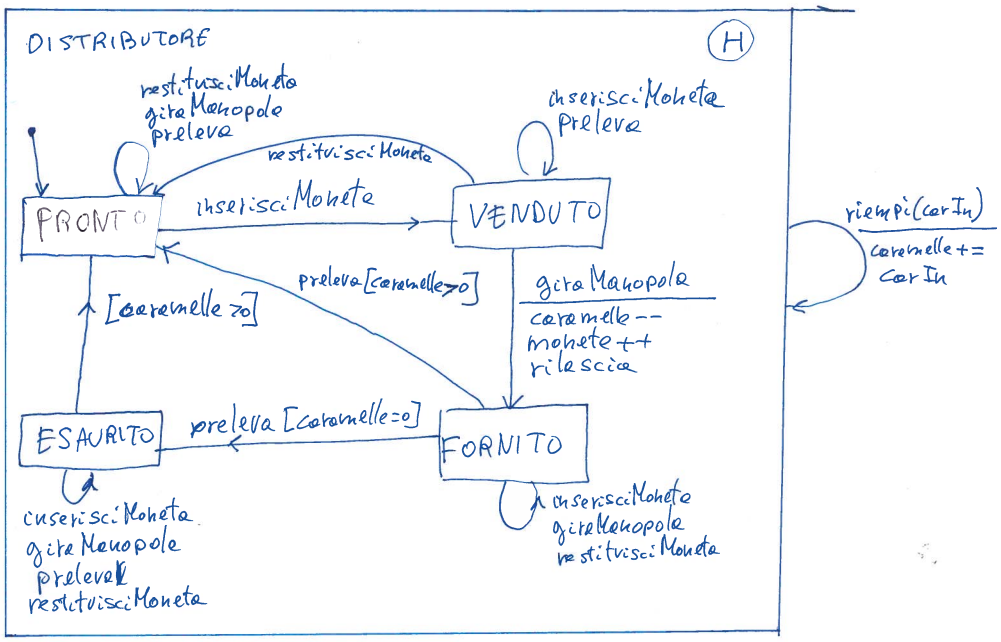
Il distributore può trovarsi nelle seguenti situazioni:

- esaurito, quando non vi è alcuna caramella;
- pronto, quando vi sono caramelle e il sistema è pronto a ricevere una moneta;
- venduto, quando l'acquirente ha inserito la moneta (ma non ha ancora girato la manopola);
- fornito, quando la caramella è stata rilasciata, in attesa di essere prelevata;

1. Descrivere questo sistema con un diagramma a stati UML, prevedendo opportune condizioni e azioni sulle transizioni.

   Soluzione: Si noti che ciascuno dei metodi inserisciMoneta(), restituisciMoneta(), giraManopola() e preleva() rappresenta un'azione esterna dell'utente (un evento) e pertanto non ha precondizione: se il metodo viene chiamato in uno stato errato, semplicemente non ha alcun effetto. Ad es. una chiamata di restituisciMoneta() a partire da uno stato diverso da VENDUTO, viene semplicemente ignorata. Questo è rappresentato dagli autoanelli nei vari stati. Supponiamo che l'aggiornamento del numero di monete e del numero di caramelle avvenga solo al momento di girare la manopola. Naturalmente altre ipotesi sono possibili. Infine, usiamo lo stato con storia per rappresentare la possibilita' che riempi() possa essere chiamato in qualunque stato, in modo da incrementare le caramelleIniziali senza modificare lo stato. Il passaggio da ESAURITO a PRONTO avviene solo in base alla condizione su

caramelle.



2. Completare la specifica della classe Distributore, definendo in JML la specifica dei metodi inserisciMoneta() e giraManopola(). La specifica deve soddisfare il vincolo che il numero di monete è pari alla somma delle caramelle erogate. Definire opportune abbreviazioni, se comodo.

   Soluzione: Riportiamo qui, per completezza, la specifica dei metodi inserisciMoneta(), restituisciMoneta(), giraManopola() e preleva(). La specifica è ricavabile immediatamente dal diagramma a stati precedente.

   Per brevità, definiamo come abbreviazione "inv(A)" l'espressione booleana:

```
A ==\old(A)

public class Distributore {
  ...
  //@ensures (\old(stato()==PRONTO) ==> stato()==VENDUTO);
  //@ensures \old(stato()!=PRONTO) ==> inv(stato());
  //@ensures inv(numMonete()) && inv(caramelle());
  public void inserisciMoneta();

  //@ensures (\old(stato()==VENDUTO) ==> stato()==PRONTO);
  //@ensures \old(stato()!=VENDUTO) ==> inv(stato());
  //@ensures inv(numMonete()) && inv(caramelle());
  public int restituisciMoneta();

  //@ensures (\old(stato()==VENDUTO) ==> stato()==FORNITO &&
  //@                              numMonete()==\old(numMonete())+1 &&
  //@                              caramelle()==\old(caramelle()-1);
  //@ensures (\old(stato()!=VENDUTO)==> inv(stato()) && inv(numMonete()) && inv(caramelle
  public void giraManopola();

  //@ensures inv(numMonete()) && inv(caramelle));
  //@ensures (\old(stato()==FORNITO) && caramelle()>0 ==> stato()==PRONTO);
  //@ensures (\old(stato()==FORNITO) && caramelle()==0 ==> stato()==ESAURITO);
```

```
  //@ensures (\old(stato()!=FORNITO ==> inv(stato());
  public void preleva();

}
```

3. Come potreste rappresentare in JML il vincolo precedente sul numero di monete e di caramelle: "il numero di monete deve essere pari alla somma delle caramelle erogate"?

   Soluzione: La proprietà è un tipico invariante pubblico, ma per potere esssee scritto come tale richiede di introdurre un nuovo metodo pubblico puro che restituisce il numero di caramelle all'inizio;

   ```
   ...public /*@ pure @*/ int caramelleInizio()...
   ```

   (Non si può introdurre una costante pubblica in quanto il metodo riempi() modifica il valore di caramelleInizio()). Occorre aggiungere alla postcondizione del costruttore e del metodo riempi() rispettivamente le condizioni:

   ```
   //@ensures caramelleInizio() == caramelleIniziali;
   //@ensures caramelleInizio() == nuoveCaramelle;
   ```

   Ad ogni altro metodo modificatore occorre aggiungere la postcondizione:

   ```
   //@ensures inv(caramelleInizio());
   ```

   Infine, l'invariante è:

   ```
   public invariant caramelleInizio()-caramelle() == numMonete();
   ```

4. Si consideri una possibile estensione del comportamento del Distributore, detta DistributoreConVincita, in cui si prevede un nuovo stato della macchina:

   - vincita, quando la macchina (dopo che l'acquirente ha inserito la moneta) decide (tramite un generatore di numeri causali) di fornire due caramelle anzichè una.

   È possibile definire questa estensione con una sottoclasse di Distributore? Giustificare la risposta.

   Soluzione: La risposta dipende dalla specifica JML precedente, ma se questa soddisfa l'invariante previsto sopra, allora sarà senz'altro violata almeno la regola delle proprietà. Per la particolare specifica data sopra, tuttavia, si viola anche la regola dei metodi, in quanto il metodo giraManopola() prevede una postcondizione chiaramente incompatibile con il caso di vincita.

Esercizio 3

Si consideri una soluzione alternativa in Java per l'esempio dell'esercizio 2. La classe Distributore (nella versione senza vincita) ha ancora un attributo "stato" che memorizza lo stato in cui si trova il distributore. Questo stato, non è un intero o un enumerativo, ma è un oggetto appartenente a una delle seguenti classi: Esaurito, Pronto, Venduto, Fornito. Queste classi forniscono gli stessi metodi inserisciMoneta(), restituisciMoneta(), giraManopola(), e preleva() della classe Distributore, ma specializzati per lo stato corrispondente.

In particolare, ciascun metodo di ogni classe restituisce il nuovo stato in cui si porta il sistema dopo l'esecuzione dell'azione. Inoltre, ciascun oggetto delle classi Esaurito, Pronto, Venduto, Fornito deve contenere anche un riferimento "distributore" all'oggetto della classe Distributore di cui rappresenta uno stato.

Ad es. il metodo restituisciMoneta() della classe Pronto si rifiuta di fornire una moneta all'utilizzatore, restituendo al chiamante ancora uno stato di tipo Pronto, mentre il metodo inserisciMoneta() è in grado di accettare una moneta (restituendo in tal caso uno stato di tipo Venduto).

1. Descrivere con un diagramma delle classi UML la soluzione appena prospettata, identificando, classi, metodi e, se opportuno, interfacce.

   Soluzione: NB: Quello proposto eùn esempio di un pattern, noto in letteratura con il nome di State. Occorre definire una classe astratta StatoDistributore, di cui le classi Esaurito, Pronto, ecc. sono estensioni (in alternativa, va bene anche un'interfaccia, implementata da Esaurito, Pronto, ecc.).

   ```
   abstract public class StatoDistributore {
   protected Distributore distributore;
   protected statoDistributore(Distributore d) {distributore=d;};
   abstract public StatoDistributore inserisciMoneta();
   abstract public StatoDistributore restituisciMoneta();
   abstract public StatoDistributore giraManopola();
   abstract public StatoDistributore preleva();
   }
   ```

   Occorre prevedere che la classe StatoDistributore abbia un'associazione 1-1 con il Distributore (che si trova esattamente in uno stato), che puo' essere rappresentata dicendo che ogni Distributore contiene una, e una sola, istanza di StatoDistributore.

2. Implementare (anche parzialmente) la rappresentazione (rep) e il costruttore di Distributore.

   Soluzione: Un rep di Distributore:

   ```
   private StatoDistributore stato;
   private int conta;
   private int monete;
   ```

   Il costruttore:

   ```
   public Distributore(int caramelleIniziali) {
    monete = 0;
    conta = caramelleIniziali;
      if (conta <=0) stato = new Esaurito();
      else stato = new Pronto();
   }
   ```

   Scrivere in JML un invariante di rappresentazione per Distributore. **Soluzione:**

   ```
   private invariant stato !=null && monete>=0 &&
   (conta <= 0 ==> stato instanceof Esaurito);
   ```

   Implementare infine il metodo giraManopola() delle classi Pronto e Distributore.

   Soluzione: Per completezza, riportiamo l'implementazione di giraManopola() anche per Fornito e Venduto.

   La classe Pronto deve rifiutarsi di dare la caramella:

```
public StatoDistributore giraManopola(){
  System.out.println(''Non hai inserito nessuna moneta!``);
  return this; //lo stato non cambia;
}
```

La classe Venduto deve chiamare rilascia() del distributore:

```
public StatoDistributore giraManopola(){
  distributore.rilascia();
  return new Fornito();
}
```

La classe Fornito deve rifiutarsi di dare la caramella:

```
public StatoDistributore giraManopola(){
  System.out.println(''Hai gi\`a preso la caramella!``);
  return this; //lo stato non cambia;
}
```

La classe Distributore delega semplicemente allo stato:

```
public void giraManopola(){
  stato = stato.giraManopola();
}
```

3. Come potreste aggiungere il caso della Vincita, minimizzando le modifiche al codice definito in precedenza (classi Distributore, Esaurito, Pronto, Venduto e Fornito)? Valutare brevemente la vostra soluzione.

   Soluzione: Definire un erede di Distributore non è di alcuna utilità, in quanto tale classe non considera in quale stato effettivo si trova il distributore. Conviene invece estendere StatoDistributore con una nuova classe Vincita. La classe Vincita deve chiamare due volte rilascia() del distributore:

```
public StatoDistributore giraManopola(){
   System.out.println(''Hai vinto una caramella!'');
  distributore.rilascia();
  distributore.rilascia();
  return new Fornito();
}
```

   Essenzialmente, occorre anche aggiungere nel metodo giraManopola dello stato Venduto, un generatore di numeri casuali per decidere se passare allo stato Vincita o restare in Venduto

```
public StatoDistributore giraManopola(){
if (''random test'') return (new Vincita()).giraManopola();
  ...qui come prima..;
}
```

   Questa soluzione è concettualmente "pulita" ma 1) vi costringe a modificare un metodo esistente; 2) vi porta a codice duplicato (tutti gli altri metodi di Vincita sono uguali a quelli di Venduto, Un'alternativa è introdurre, al posto della classe Vincita, una sottoclasse di Venduto, VendutoConPossibileVincita, che ridefinisce giraManopola() con la nuova versione randomizzata:

```
public StatoDistributore giraManopola(){
  if (``random test``) {
    System.out.println(''Hai vinto una caramella!'');
    distributore.rilascia();
  }
  return super.giraManopola();
```

   Occorrerà però modificare comunque il codice esistente, per introdurre (nel metodo inserisciMoneta() dello stato Pronto) la chiamata al costruttore di VendutoConPossibileVincita al posto di quello di Venduto. Questa soluzione evita le duplicazioni e riduce ulteriormente le modifiche del codice esistente, ma può essere concettualmente meno chiara.

Esercizio 4

Si consideri il metodo seguente e si generi un insieme "minimo" di casi di test che soddisfi il criterio di copertura delle diramazioni (*branch coverage*). Si trovi poi un insieme minimo per soddisfare il criterio di copertura della condizioni.

```
00 public static void mistery(int[] a) {
01      int i, j, n;
02      for (i = 1; i < a.length; i++) {
03            n = a[i];
04            j = i;
05            while (j > 0 && a[j - 1] > n) {
06                  a[j] = a[j - 1];
07                  j--;
08            }
09            a[j] = n;
10      }
11 }
```

Soluzione: Un qualunque array di due elementi in cui $a[0] > a[1]$ (corrispondente alla condizione $a[j-1] > n$ nel while alla prima iterazione del for) permette di coprire tutte le diramazioni. Non basta a coprire tutte le condizioni, in quanto con due elementi il ciclo for viene eseguito una sola volta e si esce dal ciclo while dopo una sola iterazione.

Per coprire tutte le condizioni (e quindi anche tutte le diramazioni), basta che l'array sia di 3 elementi, non ordinato, in modo da potere uscire dal while una volta con $j <= 0$ e una volta con $j > 0 \&\& a[j-1] <= n$. E' quindi sufficiente un singolo caso di test, come $a = [3\ 1\ 2]$. NB: l'algoritmo e' un semplice insertion sort.